



Introducción a Python

Documentación de Python: <https://docs.python.org/3/>

Python es un lenguaje de programación de alto nivel, interpretado, de tipado dinámico, con una sintaxis muy sencilla, es gratis y opensource. Es ampliamente utilizado en Data Science, Data Engineering y Data Analytics e incluso es dominante en el campo de la Inteligencia Artificial, por su amplio repositorio de librerías. Está escrito en el lenguaje de programación C, y su implementación oficial es **CPython**.

¿Qué beneficios ofrece Python?

Los beneficios de Python incluyen los siguientes:

- Los desarrolladores pueden leer y comprender fácilmente los programas de Python debido a su sintaxis básica similar a la del inglés.
- Python permite que los desarrolladores sean más productivos, ya que pueden escribir un programa de Python con menos líneas de código en comparación con muchos otros lenguajes.
- Python cuenta con una gran biblioteca estándar que contiene códigos reutilizables para casi cualquier tarea. De esta manera, los desarrolladores no tienen que escribir el código desde cero.
- Los desarrolladores pueden utilizar Python fácilmente con otros lenguajes de programación conocidos, como Java, C y C++.
- La comunidad activa de Python incluye millones de desarrolladores alrededor del mundo que prestan su apoyo. Si se presenta un problema, puede obtener soporte rápido de la comunidad.
- Python se puede trasladar a través de diferentes sistemas operativos de computadora, como Windows, macOS, Linux y Unix.



¿Cuáles son las características de Python?

Las características siguientes del lenguaje de programación Python lo hacen único:

- Un lenguaje interpretado: Python es un lenguaje interpretado, lo que significa que ejecuta directamente el código línea por línea. Si existen errores en el código del programa, su ejecución se detiene. Así, los programadores pueden encontrar errores en el código con rapidez.
- Un lenguaje fácil de utilizar: Python utiliza palabras similares a las del inglés. A diferencia de otros lenguajes de programación, Python no utiliza llaves. En su lugar, utiliza sangría.
- Un lenguaje de tipado dinámico: Los programadores no tienen que anunciar tipos de variables cuando escriben código porque Python los determina en el tiempo de ejecución. Debido a esto, es posible escribir programas de Python con mayor rapidez.
- Un lenguaje de alto nivel: Python es más cercano a los idiomas humanos que otros lenguajes de programación. Por lo tanto, los programadores no deben preocuparse sobre sus funcionalidades subyacentes, como la arquitectura y la administración de la memoria.
- Un lenguaje orientado a los objetos: Python considera todo como un objeto, pero también admite otros tipos de programación, como la programación estructurada y la funcional.

Tipos de datos primitivos en Python

Python incluye varios tipos de datos primitivos básicos para manejar información elemental:

- **int (Entero):** Representa números enteros, positivos o negativos, sin parte decimal. Ejemplo: edad = 25
- **float (Punto flotante):** Representa números reales o con punto decimal. Ejemplo: precio = 19.99
- **str (Cadena de texto):** Representa secuencias de caracteres encerradas entre comillas simples o dobles. Ejemplo: nombre = "Python"
- **bool (Booleano):** Representa un valor lógico que solo puede ser verdadero (True) o falso (False). Ejemplo: activo = True

Tipos de datos complejos en Python

Además de los tipos primitivos, Python ofrece estructuras de datos complejas para agrupar y gestionar colecciones de datos:

- **list (Lista):** Colección ordenada y mutable de elementos. Permite elementos duplicados. Ejemplo: frutas = ["manzana", "banana", "naranja"]
- **tuple (Tupla):** Colección ordenada e inmutable de elementos. Una vez creada, no se puede modificar. Ejemplo: coordenadas = (10.0, 20.0)
- **dict (Diccionario):** Colección de pares clave-valor, desordenada pero indexada por claves únicas. Ejemplo: persona = {"nombre": "Ana", "edad": 30}
- **set (Conjunto):** Colección desordenada de elementos únicos sin duplicados. Ejemplo: números = {1, 2, 3, 4}

Operaciones aritméticas en Python

Python incluye varios operadores aritméticos integrados para realizar operaciones matemáticas básicas y avanzadas:

- **Suma (+):** Suma dos valores. Ejemplo: $5 + 3$ es igual a 8.
- **Resta (-):** Resta el segundo valor del primero. Ejemplo: $10 - 4$ es igual a 6.
- **Multiplicación (*):** Multiplica dos valores. Ejemplo: $4 * 2$ es igual a 8.
- **División (/):** Divide el primer valor por el segundo y devuelve un número flotante. Ejemplo: $15 / 2$ es igual a 7.5.
- **División entera (//):** Divide el primer valor por el segundo y redondea hacia abajo al entero más cercano. Ejemplo: $15 // 2$ es igual a 7.
- **Módulo (%):** Devuelve el resto de la división. Ejemplo: $15 \% 2$ es igual a 1.
- **Potencia (**):** Eleva el primer valor a la potencia del segundo. Ejemplo: $2 ** 3$ es igual a 8.

Ejemplo de código:

Python

```
a = 10  
b = 3
```

```
print(f"Suma: {a + b}")           # 13  
print(f"Resta: {a - b}")          # 7  
print(f"Multiplicación: {a * b}") # 30  
print(f"División: {a / b}")        # 3.3333...  
print(f"División entera: {a // b}") # 3  
print(f"Módulo: {a % b}")          # 1  
print(f"Potencia: {a ** b}")       # 1000
```

Condicionales en Python (if, elif, else)

Las estructuras condicionales permiten ejecutar bloques de código específicos según se cumplan o no determinadas condiciones lógicas:

- **if:** Evalúa si una condición inicial es verdadera para ejecutar un bloque de código.
- **elif:** Permite evaluar condiciones adicionales en orden si la condición anterior no se cumplió.
- **else:** Ejecuta un bloque de código alternativo cuando ninguna de las condiciones anteriores es verdadera.

Ejemplo básico:

```
Python
edad = 18
if edad < 18:
    print("Menor de edad")
elif edad == 18:
    print("Justo tiene 18 años")
else:
    print("Mayor de edad")
```

Bucles en Python (for y while)

Los bucles permiten repetir un bloque de código múltiples veces:

- **for:** Se utiliza para iterar sobre una secuencia (como una lista, tupla, diccionario, conjunto o cadena de texto).
- **while:** Ejecuta un conjunto de instrucciones mientras una condición sea verdadera.



Ejemplo de bucle for con range

Python

```
for i in range(5):  
    print(i)
```

Ejemplo de bucle for recorriendo una lista:

Python

```
frutas = ["manzana", "banana", "cereza"]  
for fruta in frutas:  
    print(fruta)
```

Ejemplo de bucle while:

Python

```
contador = 0  
while contador < 3:  
    print(f"Contador: {contador}")  
    contador += 1
```